CORE JAVA

ROADMAP:

1 .Introduction to Java

- What is Java
- History of Java
- Features of Java
- Applications of Java
- Java Editions (Java SE, Java EE/Jakarta EE, Java ME)

2. Java Architecture

- JDK (Java Development Kit)
- JRE (Java Runtime Environment)
- JVM (Java Virtual Machine)
- Java compilation process
- Bytecode , Platform independence

3. JVM Internal Architecture

- Class Loader Subsystem
- Method Area
- Heap Memory, Stack Memory
- Program Counter Register
- Native Method Stack, Execution Engine, JIT Compiler
- Garbage Collector

4. Setting Up Java Development Environment

- Install JDK
- Configure JAVA_HOME, Configure PATH
- Verify installation (java -version, javac -version)
- Install IDE (IntelliJ / Eclipse / VS Code)
- Write and run first Java program

5. Java Program Structure

- Structure of Java program
- Class declaration, main() method

- System class, System.out.println()
- Compilation and execution process

6. Java Basics

- Identifiers, Keywords, Comments
- Naming conventions
- Command line argument

7. DataTypes

Primitive Data Types

- Byte, short
- Int ,long, float, double, char, Boolean

Non-Primitive Data Types

- String, Arrays
- Classes
- Interfaces

8. Variables

- Local variables, Instance variables, Static variables
- Variable scope and lifetime

9. Operators

- Arithmetic operators ,Relational operators, Logical operators, Bitwise operators
- Assignment operators, Unary operators, Ternary operator

10. Control Flow Statements

Decision Making

- If, if else, nested if, switch

Loops

- for loop, while loop, do while loop
- enhanced for loop

Jump Statements

- break, continue, return

11. Methods

- Method declaration, Method calling
- Parameters and arguments, Return types
- Method overloading , Recursion
- Varargs

12. Object Oriented Programming Basics

- Class
- Object, Object creation
- Fields and methods

13. Constructors

- Default constructor, Parameterized constructor
- Constructor overloading, Constructor chaining

14. this Keyword, static Keyword, final Keyword, Records

15. Access Modifiers

- Private, default, protected , public

18. OOP Principles

Encapsulation

Inheritance

- Single inheritance, Multilevel inheritance, Hierarchical inheritance

Sealed classes

Polymorphism

- Method overloading, Method overriding

Abstraction

- Abstract class
- Interface

19. Interfaces Deep Concepts

- Default methods, Static methods
- Functional interfaces

20. Packages

- Built-in packages, User defined packages, package keyword, import keyword

21. Arrays

- One dimensional arrays, Two dimensional arrays, Multidimensional arrays
- Jagged arrays , Array traversal
- Searching algorithms
- Sorting algorithms

22. Strings

- String class
- String methods, String comparison
- String constant pool,Immutable strings
- StringBuilder, String Buffer

23. Wrapper Classes

24. Type Casting

- Implicit casting, Explicit casting

25. Exception Handling

- Errors vs Exceptions
- Exception hierarchy
- Checked exceptions, Unchecked exceptions
- Try, catch ,finally , throw, throws
- Custom exceptions

26. Java Collections Framework

Collection Interfaces

- Iterable, Collection, List, Set, Queue, Map

List

- ArrayList, LinkedList, Vector, Stack

Set

- HashSet, LinkedHashSet, TreeSet, sorted set, Navigable set ,concurrent Hashset

Queue

- PriorityQueue, Deque ,LinkedBlocking queue ,ArrayBlocking Queue

Map

- HashMap, LinkedHashMap, TreeMap
- Hashtable,Identity and weak HashMap

Concepts

- Hashing, Load factor
- Collision handling ,List Iterator ,Concurrent Modification Exception, CopyonWrite ArrayList, Fail-safe and Fail-fast ,comparator and comparable

27. Generics

- Generic classes, Generic methods
- Bounded types, Wildcards

28. File Handling (I/O Streams)

- File class
- Byte streams ,Character streams
- FileInputStream, FileOutputStream
- FileReader, FileWriter
- BufferedReader, BufferedWriter

29. Serialization

- Serialization, Deserialization
- Scanner for user input
- Transient ,serialversionUId

30. Multithreading

Introduction

- Process vs Thread
- Multitasking

Thread Creation

- Extending Thread class,
- Implementing Runnable interface
- Lambda thread creation

Thread Lifecycle

- New, Runnable, Running, Blocked, Waiting, Timed waiting, Terminated

Thread Methods

- start(), run(), sleep(), join(), yield(), interrupt(), isAlive(), setPriority(), getPriority()
- setName(), getName()
- interrupt handling

Thread Scheduling

- Thread priority, Preemptive scheduling, Time slicing

Synchronization

- Synchronized method, Synchronized block, Static synchronization
- Object level lock v/s class -level lock intrinsic lock
- Race condition ,critical sections, atomic operations ,visibility issues

Inter Thread Communication

- wait(), notify(), notifyAll()
- producer and consumer problem

Thread Problems

- Deadlock -causes and detection techniques ,prevention strategies
- Starvation, Livelock

Daemon Threads

Volatile Keyword : Threadlocal, memory visibility

Thread Safety

Synchronous and Asynchronous programming

Java Concurrency Utilities (java.util.concurrent)

- Executor, ExecutorService, ThreadPoolExecutor ,Rejection Policies
- Callable, Future

Locks

- ReentrantLock, ReadWriteLock

Concurrent Collections

- ConcurrentHashMap, CopyOnWriteArrayList

Synchronizers

- CountDownLatch, CyclicBarrier, Semaphore, Phaser

Fork Join Framework

- ForkJoinPool, RecursiveTask, RecursiveAction, Work Stealing

CompletableFuture

31. Java 8 Features

- Lambda expressions, Functional interfaces
- Stream API ,parallel streams
- Method references, Optional class
- Date and Time API

32. Miscellaneous

- Enum – enum fields ,methods,constructors enum in switch ,value and valueof()
- Clone() ad cloneable() – shallow copy
- Annotation built in ,Asserions ,class loader

33. JDBC (Database Connectivity)

- JDBC architecture
- JDBC drivers
- Connection, Statement, PreparedStatement, Callable Statement
- ResultSet, Transactions

33. Networking

- Socket programming, URL class, HTTP connections

35. Build Tools

- Maven, Gradle

36. Testing

- Unit testing, Junit, Mockito

37. Web Technologies with Java

- Servlets
- JSP
- MVC architecture

38. Java Frameworks

1. Spring Framework

- **Basics**
- What is Spring Framework
- Advantages of Spring
- Spring Architecture
- **Core Concepts**
- IoC (Inversion of Control), Dependency Injection, Dependency Injection Types
- Constructor Injection, Setter Injection
- **Spring Beans**
- Bean, Bean lifecycle, Bean scopes (Singleton, Prototype)
- **Configuration**
- XML configuration, Annotation configuration, Java configuration

Important Annotations

- @Component, @Service, @Repository, @Controller, @Autowired, @Configuration, Bean

- Spring Modules
- Spring Core, Spring AOP, Spring MVC
- Spring Data, Spring MVC
- MVC architecture
- Controller
- Model
- View
- DispatcherServlet

2. Spring Boot

Introduction

- What is Spring Boot
- Difference between Spring and Spring Boot
- Creating Spring Boot Project
- Spring Initializer,
- Project structure
- **Configuration**
- application. Properties
- application.yml
- **Important Features**
- Auto Configuration
- Starter dependencies
- Embedded server (Tomcat)

3. Hibernate / JPA

- **Basics**
- What is ORM
- What is Hibernate,What is JPA
- **Core Components**
- Configuration
- SessionFactory, Session, Transaction
- **Entity Mapping**
- @Entity, @Table, @Id, @Column
- Primary Key Generation
- @GeneratedValue
- **CRUD Operations**
- Save, Update
- Delete ,Fetch
- **Relationships**
- One-to-One, One-to-Many, Many-to-One, Many-to-Many
- **Fetching**
- Eager Fetch, Lazy Fetch
- **Queries**
- HQL, JPQL
- Native SQL
- **Caching**
- First Level Cache, Second Level Cache

Spring Boot + Hibernate

- Repository Layer
- CrudRepository, JpaRepository
- Common Annotations
- @Entity, @Repository, @Service, @Transactional

4.REST API Development

Basics

- What is REST
- REST Architecture
- HTTP methods (GET, POST, PUT, DELETE)

Annotations

- @RestController, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping

Request Handling

- @RequestBody, @ResponseBody, @PathVariable, @RequestParam

Other Concepts

- ResponseEntity, Exception Handling, Validation, Logging, Profiles, Actuator

7. Microservices with Spring Boot

Basics

- What are Microservices
- Monolithic vs Microservices

Core Concepts

- Service communication
- REST APIs between services

Tools & Components

- Spring Cloud, API Gateway, service Discovery, Load Balancing

Common Technologies

- Eureka Server
- Feign Client
- Config Server

39. Advanced Java Concepts

- Design patterns
- Microservices architecture
- Security (JWT, OAuth)
- Docker

40. Developer Tools

- Git
- GitHub
- Postman

41. Project Development

- Console applications
- File handling projects
- Database applications
- REST API backend projects
- Full backend applications

<u>Java</u>

Introduction to java:

A language is a communication media.

Any language contains two important things

1) Syntax (Rules)

2) Semantics (Structure OR Meaning)

English langugae translation :

Subject + verb + Object (Syntax)

He is a boy. (Valid)

He is a box. (Invalid)

Example of Progamming Language :

```
int a = 10;

int b = 0;

int c = a/b;   //Exception
```

Note :-

Syntax of the programming language is taken care by compiler.

compiler generates errors if a user does not follow the syntax of the programming language.

Semantics is taken care by our runtime Environment. It generates Exception if a user does not follow the semantics.

Statically(Strongly) typed language :-

The languages where data type is compulsory before initialization of a variable are called statically typed language.In these languages we can hold same kind of value during the execution of the program.

Ex:- C,C++,Core Java, C#

Dynamically(Loosly) typed language :-

The languages where data type is not compulsory and it is optional before initialization of a variable then it is called dynamically typed language.

In these languages we can hold different kind of value during the execution of the program.

Ex:- Visual Basic, Javascript, Python

Flavors Of JAVA language :

---

1) JSE (Java Standard Edition) ->J2SE -> Core Java

2) JEE (Java Enterprise Edition) -> J2EE -> Advanced Java

3) JME (Java Micro Edition) -> J2ME -> Android Application

4) Java FX [out dated by using we can create GUI applications]

What is the difference between stand-alone programs and web-related programs?

Standalone Application-

If the creation(development), compilation and execution of the program, everything is done in a single system then it is called stand-alone program.

Eg:- C, C++, Java, C# and so on.

Stand alone programs are also known as Software OR Desktop application.

Web - related Application :-

If creation of the program, compilation of the program and execution of the program, Everything is done on different places then it is called web related program.

Eg:- Advanced Java, PHP, ASP.NET, Python

Web related programs are also known as websites or web application.

What is a function :-

A function is a self defined block for any general purpose, calculation or printing some data.

The major benefits with function are :-

---

1) Modularity :- Dividing the bigger modules into number of smaller modules where each module will perform its independent task.

2) Easy understanding :- Once we divide the bigger task into number of smaller tasks then it is easy to understand the entire code.

3) Reusability :- We can reuse a particular module so many number of times so It enhances the reusability nature.

Note :- In java we always reuse our classes.(WORA)

4) Easy Debugging :- Debugging means finding the errors, With function It is easy to find out the errors because each module is independent with another module.

Why we pass parameter to a function :-

We pass parameter to a function for providing more information regrading the function.

Eg:-

```
public void deposit(int amount)
{
}

public void sleep(int hours)
{
}
```

Why functions are called method in java?

In C++ there is a facility to write a function inside the class as well as outside of the class by using :: (Scope resolution Operator), But in java all the functions must be declared inside the class only.

That is the reason member functions are called method in java.

Variable --> Field

function ---> Method

The role of compiler :

---

a) Compilers are used to check the syntax.

b) It also check the compatibility issues(LHS = RHS)

c) It converts the source code into machine code.

Some imporatnt points to remember :

---

a) Java programs must be saved having extension .java

b) java compiler(javac) is used to compile our code.

c) After successful compilation we are getting .class file (bytecode)

d) This .class file we submit to JVM for execution prupose (for executing my java code)

JVM :- It stands for Java Virtual Machine. It is a software in the form of interpreter written in 'C' language.

Every browser contains JVM, Those browsers are known as JEB (Java Enabled Browsers) browsers.

Why java language is so popular in the It industry :-

C and C++ programs are platform dependent programs that means the .exe file created on one machine will not be executed on the another machine if the system configuration is different.

That is the reason C and C++ programs are not suitable for website development.

Where as on the other hand java is a platform independent language. Whenever we write a java program, the extension of java program must be .java. Now this .java file we submit to java compiler (javac) for compilation process. After successful compilation the compiler will generate a very special machine code file i.e .class file (also known as bytecode). Now this .class file we submit to JVM for execution purpose.

The role of JVM is to load and execute the .class file. Here JVM plays a major role because It converts the .class file into appropriate machine code instruction (Operating System format) so java becomes platform independent language and it is highly suitable for website development.

Note :- We have different JVM for different Operating System that means JVM is platform dependent technology where as Java is platform Independent technology.

What is the difference between the bit code and byte code.

Bit code is directly understood by Operating System but on the other hand byte code is understood by JVM, JVM is going to convert this byte code into machine understandable format (Operating System format).

What is Java?

Java is a high-level, object-oriented, platform-independent programming language used to develop a wide variety of applications such as:

- Web applications
- Mobile applications
- Desktop software
- Enterprise systems
- Cloud-based applications
- Embedded systems

Java is designed to be simple, secure, robust, and portable, allowing developers to write code once and run it anywhere.

History of Java

Java was developed by James Gosling and his team at Sun Microsystems in 1995.

Originally, Java was created for consumer electronic devices, but later it became popular for internet and enterprise applications.

In 2010, Oracle Corporation acquired Sun Microsystems and became the official maintainer of Java.

First Name of Java : OAK (In the year 1991 which is a tree name)

Project Name :- Green Project

Inventor of Java : - James Gosling and his friends

Official Symbol :- Coffee CUP

Java :- Island (Indonesia)

Birthday :- 23rd January 1996 [JDK 1.0]

Why Java?

Java became one of the most popular programming languages because of its powerful features:

1. Platform Independent

Java follows the principle:

Write Once, Run Anywhere (WORA)

Java programs are compiled into **bytecode**, which can run on any system that has a **Java Virtual Machine (JVM)**.

This is achieved through the **Java Virtual Machine (JVM)**.

Process:

Source_Code-(.java)
↓
Compiler-(javac)
↓
Bytecode(.class)
↓
Java_Virtual_Machine(JVM)
↓
Machine Code

This concept is known as **Write Once, Run Anywhere (WORA)**.

---

2. Object-Oriented

Java follows **Object-Oriented Programming (OOP)** principles such as:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

These principles help build **modular, reusable, and maintainable code**.

---

3. Simple

Java syntax is similar to **C and C++**, but it removes complex features like:

- Pointer arithmetic
- Multiple inheritance with classes
- Manual memory management

This makes Java easier to learn and use.

---

4. Secure

Java provides a **secure execution environment** because:

- No direct memory access
- Bytecode verification

- Security manager

This makes Java suitable for **network and enterprise applications**.

---

5. Robust

Java is robust because it includes:

- Strong memory management
- Garbage collection
- Exception handling

These features reduce crashes and improve reliability.

---

6. Multithreaded

Java supports **multithreading**, allowing multiple tasks to run simultaneously within a program.

Example uses:

- Web servers
- Real-time applications
- High-performance systems

---

7. Distributed

Java provides built-in support for **network programming**, enabling applications to run across multiple systems.

8. High Performance

Java uses **Just-In-Time (JIT) compilation** which converts bytecode into native machine code at runtime, improving program execution speed.

---

9. Portable

Java programs can run on different systems because Java bytecode is platform-independent and executed by the JVM.

---

10. Dynamic

Java supports **dynamic class loading**, meaning classes can be loaded during runtime when needed.

Java Applications

Java is used in many types of applications:

Web Applications

Frameworks like:

- Spring, Hibernate

Mobile Applications

Android apps are primarily developed using Java.

Desktop Applications

Java libraries like **Swing** and **JavaFX** help build graphical applications.

Enterprise Applications

Large-scale systems like banking and e-commerce platforms use Java.

6. Big Data Technologies

Many big data tools are built using Java, including:

- Apache Hadoop, Apache Spark

These technologies process massive datasets in distributed systems.

---

7. Cloud Applications

Java is widely used to develop cloud-based services and microservices architectures.

---

8. Internet of Things (IoT)

Java can be used in embedded systems and IoT devices due to its portability and security.

Java Editions

Java is divided into different editions to support different types of applications.

---

1. Java SE (Java Standard Edition)

Java Standard Edition

Java SE provides the **core Java platform** used for general-purpose programming.

It includes:

- Basic language features, Core libraries, JVM, Development tools

Common uses:

- Desktop applications, Core Java programs, Basic server applications

Important packages include:

- java.lang , java.util, java.io, java.net

Java SE is the **foundation for all other Java editions**.

---

2. Java EE / Jakarta EE

Jakarta EE (previously **Java EE**)

Java EE extends Java SE with additional APIs for building **large-scale enterprise applications**.

It provides technologies such as:

- Servlets
- JSP (Java Server Pages), Enterprise JavaBeans (EJB), Web services, REST APIs

Java EE is widely used for:

- Banking systems, Enterprise software, Distributed applications

Originally developed by **Sun Microsystems**, it is now maintained by the **Eclipse Foundation** under the name Jakarta EE.

---

3. Java ME (Java Micro Edition)

Java Micro Edition

Java ME is designed for **small devices with limited resources**.

Examples include:

- Mobile phones (early generation), Embedded systems, IoT devices, Set-top boxes

Java ME provides a **lightweight Java platform** optimized for devices with limited memory and processing power.

Edition	Purpose
Java SE	Core Java platform
Java EE / Jakarta EE	Enterprise and web applications
Java ME	Embedded and mobile devices

Java Architecture

Java architecture explains how Java programs are **developed, compiled, and executed**. The Java platform is designed in a layered architecture consisting of:

- **JDK (Java Development Kit)**
- **JRE (Java Runtime Environment)**
- **JVM (Java Virtual Machine)**

These components work together to compile and run Java programs on different platforms.

JDK (Java Development Kit)

Java Development Kit

The **JDK** is a complete development environment used for creating Java applications.

It includes everything needed for **developing, compiling, and running Java programs**.

Java Development Kit (JDK) is a cross-platform software development kit that provides tools and libraries needed to build Java-based applications and applets. It works together with the JVM (Java Virtual Machine) and JRE (Java Runtime Environment) as part of the core Java setup.

- The JDK includes everything required for Java development — compilers, debuggers, and other essential tools.
- Beginners often get confused between JRE and JDK.
- If you only want to run Java programs, installing JRE is enough.
- But if you want to develop Java applications, you'll need the JDK, which includes the JRE and additional development tools.

JDK = JRE + Development Tools

Components of JDK

The JDK mainly contains:

1. **JRE**
2. Development tools
3. Java libraries

Important tools included in JDK:

Tool	Purpose
`javac`	Compiles Java source code into bytecode
`java`	Runs the Java program
`javadoc`	Generates documentation
`jar`	Packages Java files into JAR archives
`jdb`	Java debugger

JDK Structure

JDK

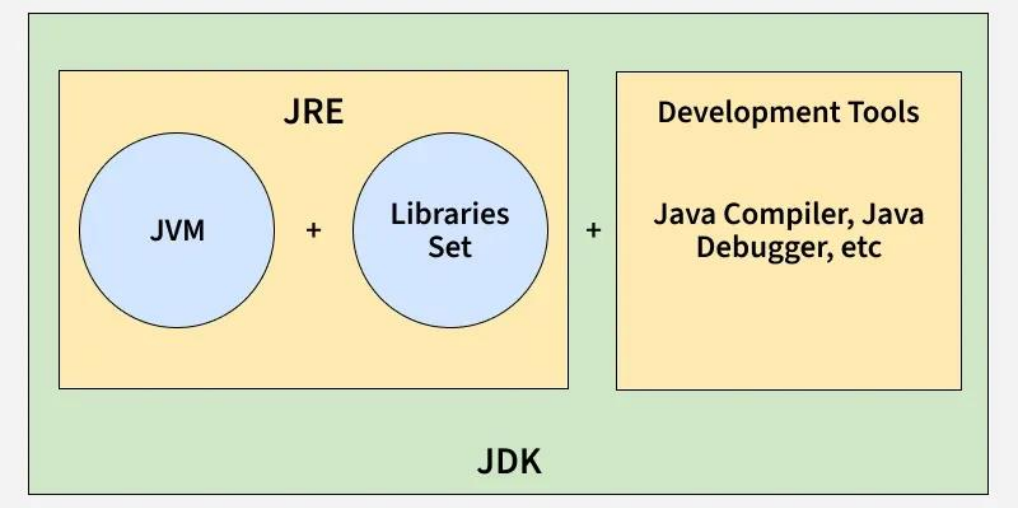
├── Development Tools

├── JRE

│ ├── JVM

│ └── Core LibrariesSo we can say:

JDK = JRE + Development Tools



JRE (Java Runtime Environment)

Java Runtime Environment

The **JRE** provides the environment required to **run Java programs**.

It does **not contain development tools** like compilers.

It is the part of the Java platform responsible for **running** Java applications — not for developing them.

1. What is JRE?

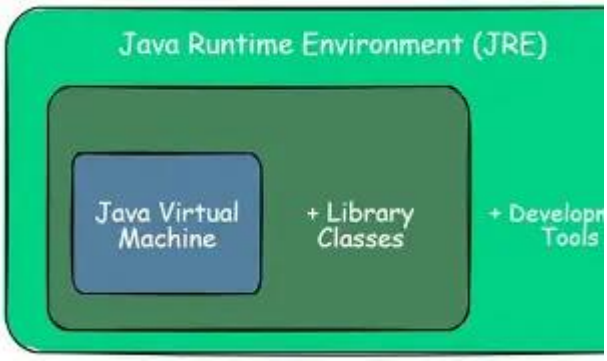
- **Definition**: A software package that provides the **libraries**, **Java Virtual Machine (JVM)**, and other components needed to run Java programs.
- **Purpose**: It does **not** contain development tools like compilers or debuggers (those are in the **JDK**).
- **Analogy**: If Java is a language, the JRE is like the "player" that can read and execute the "script" (Java bytecode).

2. Components of JRE

1. **JVM (Java Virtual Machine)**
    - Executes Java bytecode.
    - Handles memory management, garbage collection, and security.
2. **Java Class Libraries**
    - Pre-built classes (e.g., java.lang, java.util, java.io) that Java programs use.
3. **Supporting Files**
    - Configuration files, property files, and resource files.

4. When to Use JRE

- If you **only need to run** Java applications (not develop them).
- Example: Running a Java-based desktop app like Eclipse or Minecraft.



JRE = JVM + Libraries

Instead, it includes:

- JVM
- Core class libraries, Supporting files

Components of JRE

1. **JVM (Java Virtual Machine)**
2. Java standard libraries
3. Runtime support files

Relationship:

JRE = JVM + Libraries

The JRE ensures that Java programs can run on different operating systems.

JVM (Java Virtual Machine)

Java Virtual Machine

The **JVM** is an abstract machine responsible for **executing Java bytecode**.

It acts as a bridge between **Java programs and the operating system**.

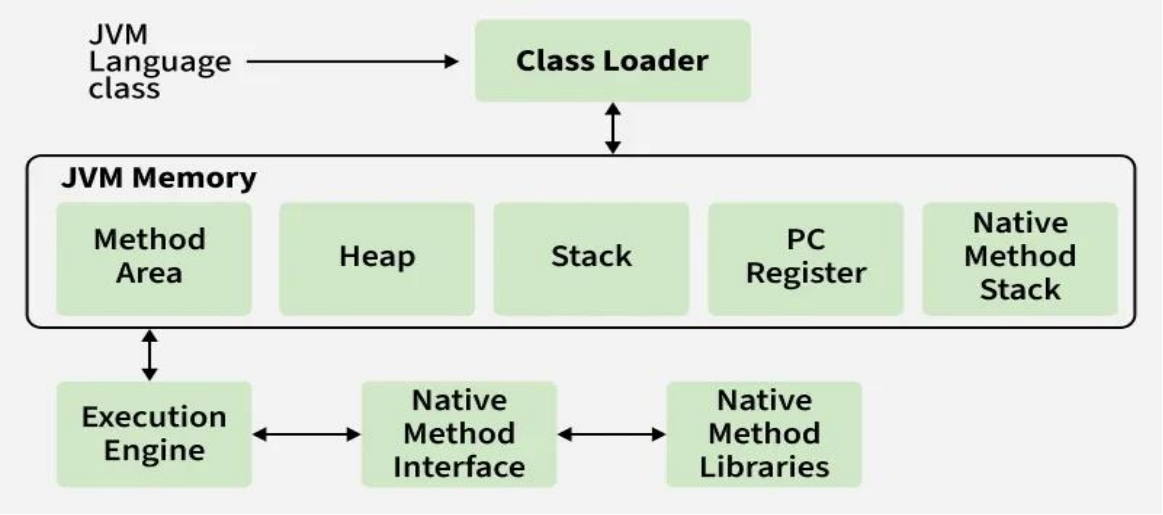
he Java Virtual Machine (JVM) is a core component of the Java Runtime Environment (JRE) that allows Java programs to run on any platform without modification. JVM acts as an interpreter between Java bytecode and the underlying hardware, providing Java's famous Write Once, Run Anywhere (WORA) capability.

- Java source (.java) -> compiled by javac -> bytecode (.class)
- JVM loads the bytecode, verifies it, links it, and then executes it
- Execution may involve interpreting bytecode or using Just-In-Time (JIT) compilation to convert "hot" code into native machine code for performance
- Garbage collection runs in the background to reclaim memory from unused objects

The JVM performs several important tasks:

- Loads class files
- Verifies bytecode
- Executes program instructions
- Manages memory
- Performs garbage collection

JVM Architecture



Main components of JVM:

JVM

```
├── Class Loader
├── Runtime Data Areas
│   ├── Method Area
│   ├── Heap
│   ├── Stack
│   ├── PC Register
│   └── Native Method Stack
├── Execution Engine
└── Garbage Collector
```

Key Responsibilities of JVM

- Bytecode execution
- Memory management
- Security
- Platform independence

Java Compilation Process

The **Java compilation process** is the sequence of steps that converts Java source code into executable instructions that a computer can run.

Unlike many programming languages, Java uses a **two-step execution process** involving **compilation and interpretation**.

Steps in Java Compilation

1. Writing the Source Code

The programmer writes a Java program using a text editor or IDE. The file is saved with the **.java extension**.

- You write source code in a .java file.
- The **Java Compiler (javac)** translates this source code into **Bytecode**.
- Bytecode is stored in .class files.
- **Bytecode** is **not machine-specific** — it's an intermediate, platform-independent representation

Example:

```
public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello Java");
  }
}
```

File name:

HelloWorld.java

---

2. Compilation by Java Compiler

The **Java compiler (javac)** compiles the source code into **bytecode**.

Command:

```
javac HelloWorld.java
```

Output:

HelloWorld.class

This .class file contains **bytecode instructions**.

The compiler also checks for:

- Syntax errors
- Type checking
- Program structure

If errors exist, the program will not compile.

---

3. Class Loading

The **Java Virtual Machine** loads the compiled .class file into memory using the **Class Loader**.

The Class Loader performs tasks such as:

- Loading class files
- Linking classes
- Initializing static variables

---

4. Bytecode Verification

Before execution, the JVM verifies the bytecode to ensure:

- It does not violate security rules
- It follows proper Java standards
- Memory access is safe

This step prevents malicious code from executing.

---

5. Execution by JVM

The JVM executes the bytecode using its **execution engine**.

It converts bytecode into **native machine code** that the operating system understands.

- The **Java Virtual Machine (JVM)** reads the .class file.
- The JVM converts Bytecode into **machine code** for the specific operating system and CPU.
- This conversion is done by:
    - **Interpreter** (executes bytecode line-by-line)
    - **JIT Compiler** (Just-In-Time compiler) — compiles bytecode into native code for faster execution.

Java Compilation Flow

Java Source Code (.java)

↓

Java Compiler (javac)

↓

Bytecode (.class)

↓

Java Virtual Machine (JVM)

↓

Machine Code

↓

Program Output

Bytecode

Bytecode is the **intermediate code generated by the Java compiler**.

It is stored in **.class files** and executed by the **Java Virtual Machine (JVM)**.

Unlike machine code, bytecode is **not specific to any operating system or hardware**.

Characteristics of Bytecode

- Platform independent
- Compact and efficient
- Secure
- Executed by JVM

Example:

```
public class Hello {
  public static void main(String[] args) {
    System.out.println("Hello World");
  }
}
```

HelloWorld.java → HelloWorld.class

The .class file contains bytecode instructions such as:

getstatic
ldc
invokevirtual
return

These instructions are understood by the JVM.

Platform Independence

One of the most important features of Java is **platform independence**.

A Java program written on one platform can run on another platform without modification.

- Java is **"Write Once, Run Anywhere" (WORA)**.
- This is possible because:
    1. **Java Compiler** → Produces **Bytecode** (same for all platforms).
    2. **JVM** → Platform-specific implementation that converts bytecode to native code.
- Every OS (Windows, Linux, macOS, Android, etc.) has its own JVM implementation, but **all understand the same bytecode**.

This concept is known as:

Write Once, Run Anywhere (WORA).

How Platform Independence Works

Java achieves platform independence through **bytecode and JVM**.

Steps:

1. Java source code is compiled into **bytecode**.
2. Bytecode is **not tied to any specific operating system**.
3. The **JVM on each system converts bytecode into machine code**.

Example:

Java Source (.java)

|

Java Compiler (javac)

|

Bytecode (.class) <-- Same for all platforms

|

Platform-specific JVM

|

Native Machine Code

Thus, the same .class file can run on different systems.

Role of JVM in Platform Independence

The **Java Virtual Machine** acts as a **translator between bytecode and machine code**.

Each operating system has its own JVM implementation, but they all understand the same bytecode.

For example:

Operating System	JVM Implementation
Windows	Windows JVM
Linux	Linux JVM
macOS	Mac JVM

Because of JVM, the **Java program does not depend on the operating system**.

Execution flow :

.java → javac → .class → JVM → Machine Code

JVM Internal Architecture

Java Virtual Machine

The **JVM (Java Virtual Machine)** is the core component of the Java platform responsible for **executing Java bytecode**. It provides a runtime environment where Java programs can run independently of the underlying operating system.

The JVM performs several key functions:

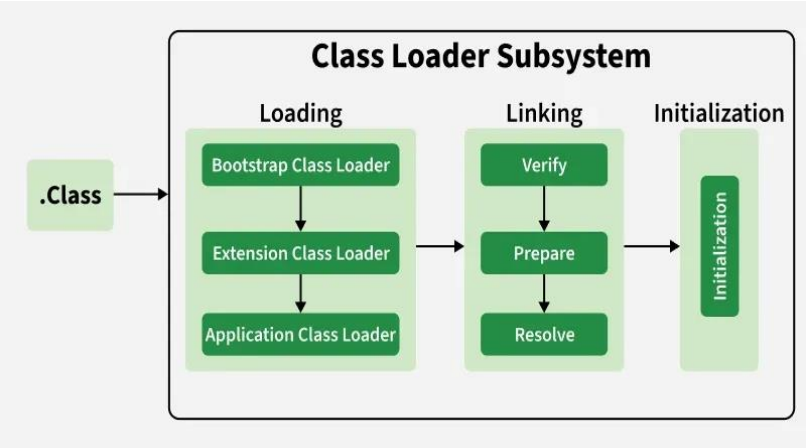
- Loads Java class files
- Verifies bytecode
- Executes program instructions
- Manages memory
- Performs garbage collection

High-Level JVM Architecture

```
                JVM Architecture
-----------------------------------------------
|            Class Loader Subsystem            |
-----------------------------------------------
|              Runtime Data Areas              |
|  Method Area | Heap | Stack | PC Register    |
|               Native Method Stack            |
-----------------------------------------------
|               Execution Engine               |
|           Interpreter | JIT Compiler         |
-----------------------------------------------
|              Garbage Collector               |
-----------------------------------------------
|              Native Method Interface         |
|              Native Libraries                |
-----------------------------------------------
```

1. Class Loader Subsystem

It is mainly responsible for three activities.



The **Class Loader Subsystem** is responsible for **loading Java class files into memory**.

When a Java program runs, the JVM does not load all classes at once. Instead, classes are loaded **dynamically when they are needed**.

Functions of Class Loader

1. **Loading**
2. **Linking**
3. **Initialization**

Loading Phase

The JVM loads the .class file from different sources:

- File system
- Network
- JAR files
- Database

Classes are loaded into **Method Area**.

Linking Phase

Linking consists of three steps:

1. Verification

Ensures the bytecode is valid and secure. Byte code follows jvm rules and is safe to execute

Checks include:

- Correct bytecode format
- No illegal memory access
- Proper instruction usage

---

2. Preparation

Memory is allocated for **static variables** and default values are assigned.

Example:

static int x = 10;

Initially stored as:

x = 0

---

3. Resolution

Symbolic references are replaced with **direct references in memory**.

Initialization

Static variables are assigned actual values and **static blocks execute**.

- Assigns actual values to static variables.
- Executes static blocks defined in the clas

Example:

static int x = 10;

Now value becomes **10**.

Types of Class Loaders

Java uses three main class loaders:

- **Bootstrap Class Loader:** Loads core Java classes (JAVA_HOME/lib).
- **Extension Class Loader:** Loads classes from extensions directory (JAVA_HOME/jre/lib/ext).
- **System/Application Class Loader:** Loads classes from the application classpath.

1. Bootstrap Class Loader

Loads core Java classes such as:

- java.lang
- java.util
- java.io

These classes come from the Java standard library.

---

2. Extension Class Loader

Loads classes from the **extension libraries**.

---

3. Application Class Loader

Loads classes from the **application classpath**.

---

2. Runtime Data Areas

The JVM uses several memory areas during program execution.

These are called **Runtime Data Areas**.

They include:

- **Method Area:** Stores class-level information like class name, parent class, methods, variables, and static data. Shared across the JVM.
- **Heap Area:** Stores all objects. Shared across the JVM.
- **Stack Area:** Each thread has its own runtime stack; stores method calls, local variables in stack frames. Destroyed when the thread ends.
- **PC Registers:** Hold the address of the currently executing instruction for each thread.
- **Native Method Stacks:** Each thread has a separate stack for native method execution.

Method Area

The **Method Area** stores **class-level information**.

Stored data includes:

- Class metadata
- Method data
- Static variables
- Runtime constant pool
- Method bytecode

Example:

```
class Test {
   static int x = 10;
}
```

x is stored in the **Method Area**.

The Method Area is **shared among all threads**.

Heap Memory

The **Heap** is the main memory area where **objects are created**. The **Heap** is the main memory area used for storing **objects and instance variables**.

Example:

```
Student s = new Student();
```

The object Student is stored in the **Heap memory**.

Characteristics:

- Shared among all threads
- Managed by the JVM
- Automatically cleaned using Garbage Collector

Heap Structure (Simplified)

```
Heap
 ├── Young Generation
 │    ├── Eden
 │    ├── Survivor S0
 │    └── Survivor S1
 │
 └── Old Generation
```

New objects are created in **Eden space**.

Young Generation

New objects are created in **Eden Space**.

If objects survive garbage collection, they move to **Survivor spaces**.

Old Generation

Objects that live longer are moved to the **Old Generation**.

These objects are considered long-lived.

Examples:

- Application caches
- Database connections
- Shared objects

Stack Memory

Each **thread has its own stack memory**.

The stack stores:

- Method calls
- Local variables
- Partial results

Example:

```
void add() {
  int a = 10;
  int b = 20;
}
```

Variables a and b are stored in the **stack frame**.

When the method finishes, the stack frame is removed.

Program Counter (PC) Register

Each thread has a **Program Counter Register**.

It stores the **address of the current instruction being executed**.

Example:

If JVM is executing instruction number 25:

PC Register = 25

Purpose:

- Keeps track of program execution
- Supports multithreading

Native Method Stack

The **Native Method Stack** supports **native methods** written in languages like:

- C
- C++

Example:

```
public native void readFile();
```

Native methods interact with **native system libraries**.

Execution Engine

Execution engine executes the ".class" (bytecode). It reads the byte-code line by line, uses data and information present in various memory area and executes instructions. It can be classified into three parts:

- **Interpreter:** It interprets the bytecode line by line and then executes. The disadvantage here is that when one method is called multiple times, every time interpretation is required.

  Bytecode → Interpret → Execute

- **Just-In-Time Compiler(JIT):** It is used to increase the efficiency of an interpreter. It compiles the entire bytecode and changes it to native code so whenever the interpreter sees repeated method calls, JIT provides direct native code for that part so re-interpretation is not required, thus efficiency is improved.

  Bytecode → JIT Compiler → Native Machine Code

- **Garbage Collector:** It destroys un-referenced objects. For more on Garbage Collector,

Complete JVM Execution Flow

```
Plain text

Java Source Code (.java)
        ↓
Compiler (javac)
        ↓
Bytecode (.class)
        ↓
Class Loader
        ↓
Runtime Data Areas
        ↓
Execution Engine
        ↓
JIT Compilation
        ↓
Program Execution
        ↓
Garbage Collector cleans unused objects
```

4. Java Native Interface (JNI)

It is an interface that interacts with the Native Method Libraries and provides the native libraries(C, C++) required for the execution. It enables JVM to call C/C++ libraries and to be called by C/C++ libraries which may be specific to hardware.

5. Native Method Libraries

These are collections of native libraries required for executing native methods. They include libraries written in languages like C and C++.

Garbage Collector:

Garbage collection in Java is an automatic memory management process that helps Java programs run efficiently. In Java, memory management is handled automatically by the **Java Virtual Machine** using a mechanism called **Garbage Collection (GC)**.

The **Garbage Collector** automatically identifies and removes objects that are **no longer reachable** in memory, freeing heap space for new objects.This eliminates the need for manual memory management as seen in languages like C or C++.

- Objects are created on the heap area.
- Eventually, some objects will no longer be needed.
- Garbage collection is an automatic process that removes unused objects from heap.

Working of Garbage Collection

- It identifies which objects are still in use (referenced) and which are not in use (unreferenced).
- It removes the objects that are unreachable (no longer referenced).
- The programmer does not need to mark objects to be deleted explicitly. Garbage collection is implemented within the JVM.

Types of Activities in Java Garbage Collection

Java heap is divided into generations:

- **Young Generation:** In this new objects are allocated.
- **Old Generation:** In this long-lived objects are stored.

Two types of garbage collection activities usually happen in Java. These are:

- **Minor or incremental Garbage Collection (GC)**: This occurs when unreachable objects in the Young Generation heap memory are removed.
- **Major or Full Garbage Collection (GC)**: This happens when objects that survived minor garbage collection are removed from the Old Generation heap memory. It occurs less frequently than minor garbage collection.

Reachability Concept (Core GC Principle)

The JVM determines object eligibility using **reachability analysis**.

Objects are considered **alive if reachable from GC Roots**.

GC Roots include:

- Local variables in stack
- Static variables
- Active threads

- JNI references

Key Concepts of Garbage Collection in Java

Garbage Collection is a process in which the **Java Virtual Machine** automatically removes unused objects from memory.

It helps manage memory efficiently and prevents memory leaks.

---

1. Unreachable Objects

An **unreachable object** is an object that **no longer has any reference pointing to it**.

When an object has no reference, the program cannot access it anymore, so it becomes eligible for garbage collection.

Example

```
Integer i = new Integer(4);

// object is reachable

i = null;

// object becomes unreachable
```

Explanation:

- Initially, the object 4 is referenced by i.
- After i = null, no reference points to the object.
- The object becomes **eligible for garbage collection**.

---

2. Making Objects Eligible for Garbage Collection

An object becomes **eligible for garbage collection when it becomes unreachable**.

Although Java automatically manages memory, programmers should **remove references to unused objects** so the Garbage Collector can clean them.

Ways to Make an Object Eligible for GC

1. Nullifying the Reference

Setting the reference to null.

```
Student s = new Student();

s = null;
```

The object becomes unreachable.

---

2. Reassigning the Reference

Assigning the reference variable to another object.

```
Student s = new Student();

s = new Student();
```

The first object becomes unreachable.

---

3. Object Created Inside a Method

Objects created inside a method become unreachable when the method finishes execution.

```
void test() {

   Student s = new Student();

}
```

After the method ends, s disappears and the object becomes eligible for GC.

---

4. Island of Isolation

This happens when objects reference each other but **no external reference points to them**.

Example:

```
class Test {

   Test ref;

}

Test obj1 = new Test();

Test obj2 = new Test();

obj1.ref = obj2;

obj2.ref = obj1;

obj1 = null;

obj2 = null;
```

Both objects reference each other but are **not reachable from the program**, so they become eligible for garbage collection.

---

3. Requesting Garbage Collection

Garbage collection runs automatically at the **JVM's discretion**.

We **cannot force the JVM to run GC**, but we can **request it**.

Two common ways:

Using System.gc()

System.gc();

Using Runtime Class

Runtime.getRuntime().gc();

Important note:

- These methods **only request garbage collection**.
- The JVM may ignore the request.

---

4. The finalize() Method (Deprecated)

Before Java destroys an object, the Garbage Collector used to call the finalize() method for cleanup tasks.

Example:

```
@Override
protected void finalize() throws Throwable {
   System.out.println("Cleaning object");
}
```

However, this method is **deprecated since Java 9** because:

- It is unpredictable
- It can slow down the program
- It may cause performance issues

Modern Java programs use better alternatives like:

- try-with-resources
- Explicit cleanup methods
-

Concept	Meaning
Unreachable Object	Object with no references
Eligible for GC	Object ready to be removed
System.gc()	Requests JVM to run GC
finalize()	Deprecated cleanup method

Reachability Graph

```
GC Roots
  |
  ▼
Object A
  |
  ▼
Object B
  |
  ▼
Object C
```

If **Object A loses its reference**, then B and C also become unreachable.

Garbage Collection Process

The garbage collector works in several phases.

1. Mark Phase

The JVM scans memory and **marks reachable objects**.

Reachable Objects → Marked

Unreachable Objects → Not Marked

---

2. Sweep Phase

Unmarked objects are removed from memory.

Marked Objects → Kept

Unmarked Objects → Deleted

---

3. Compact Phase

Memory is reorganized to remove fragmentation.

Before Compaction

Object A | Free | Object B | Free | Object C

After Compaction

Object A | Object B | Object C | Free Space

This improves memory efficiency.

Major Garbage Collection Algorithms

The JVM uses different algorithms for garbage collection.

---

1. Mark and Sweep Algorithm

This is the simplest GC algorithm.

Steps

1. Mark reachable objects
2. Sweep unreachable objects

Diagram

Heap Before GC

A | B | Garbage | C | Garbage | D

After GC

A | B | Free | C | Free | D

Problem

Creates **memory fragmentation**.

---

2. Mark and Compact Algorithm

Improves the previous algorithm by **compacting memory**.

Steps

1. Mark reachable objects
2. Remove garbage
3. Move objects together

Advantage

- Eliminates memory fragmentation

---

3. Copying Algorithm

Used mainly in **Young Generation**.

Memory is divided into two spaces:

From Space

To Space

Steps:

1. Copy live objects from **From Space**
2. Move them to **To Space**
3. Clear old memory

Generational Garbage Collection

Modern JVMs use **Generational GC**.

Objects are divided based on lifespan.

Heap Memory

Young Generation

  Eden

  Survivor S0

  Survivor S1

Old Generation

Young Generation GC (Minor GC)

Most objects die quickly.

Example:

```
String s = new String("Hello");
```

Temporary objects are created frequently.

Minor GC cleans **Young Generation**.

Characteristics:

- Fast
- Frequent

Old Generation GC (Major GC)

Objects that survive many GC cycles move to **Old Generation**.

Examples:

- Application caches
- Long-lived objects

Major GC cleans **Old Generation**.

Characteristics:

- Slower
- Less frequent

Full Garbage Collection

A **Full GC** cleans the entire heap.

This includes:

- Young Generation
- Old Generation
- Method area (sometimes)

Full GC can cause **application pauses**.

Modern Garbage Collectors

Modern JVMs support several advanced collectors.

Serial Garbage Collector

Uses **single thread**.

Best for small applications.

JVM option:

-XX:+UseSerialGC

Parallel Garbage Collector

Uses **multiple threads**.

Good for throughput.

-XX:+UseParallelGC

G1 Garbage Collector

The **Garbage-First Garbage Collector (G1 GC)** is designed for large heaps.

Features:

- Divides heap into regions
- Predictable pause times
- Efficient memory usage

Enabled using:

-XX:+UseG1GC

Z Garbage Collector

The **Z Garbage Collector (ZGC)** is designed for **very large heaps** with **ultra-low latency**.

Features:

- Pause times < 10 ms
- Scales to terabytes of memory

Stop-The-World (STW) Events

During garbage collection, all application threads may pause.

This is called **Stop-The-World**.

Example:

Application_Running
↓
Stop-The-World_Pause
↓

Garbage_Collection
↓
Application Resumes

Minimizing STW pauses is a major goal of modern GC algorithms.

Garbage Collection Monitoring Tools

Java provides tools to analyze GC behavior.

Examples:

- VisualVM
- JConsole
- Java Mission Control

These tools show:

- Heap usage
- GC frequency
- Memory leaks
- Thread activity

JVM GC Logging

Developers can enable GC logs.

Example:

-XX:+PrintGCDetails

This helps analyze memory behavior.

Best Practices for Efficient Garbage Collection

1. Avoid unnecessary object creation
2. Reuse objects when possible
3. Use appropriate GC algorithm
4. Monitor heap usage
5. Optimize heap size

Setting Up Java Development Environment

Before writing Java programs, you need to set up the **Java development environment** on your system. This involves installing the **Java Development Kit**, configuring environment variables, and installing an IDE.

---

1. Install JDK

The **Java Development Kit (JDK)** is the main software required to develop Java applications. It includes:

- Java compiler (javac)
- Java runtime
- Development tools
- Standard libraries

You can download the JDK from **Oracle Corporation**.

Steps to Install JDK

1. Go to the official Java download page.
2. Download the latest JDK version for your operating system.
3. Run the installer.
4. Follow the installation instructions.

Example installation location:

C:\Program Files\Java\jdk-21

---

2. Configure JAVA_HOME

JAVA_HOME is an environment variable that points to the **JDK installation directory**.

Steps (Windows)

1. Open **System Properties**
2. Click **Advanced System Settings**
3. Click **Environment Variables**
4. Create a new system variable:

Variable Name: JAVA_HOME
Variable Value: C:\Program Files\Java\jdk-21

This tells the system where Java is installed.

---

3. Configure PATH

The **PATH variable** allows the system to run Java commands from any terminal.

Add the following to PATH:

%JAVA_HOME%\bin

This enables commands like java and javac.

---

4. Verify Java Installation

After installation, verify that Java is installed correctly.

Open **Command Prompt / Terminal** and run:

Check Java version

java -version

Example output:

java version "21"

Check Java compiler

javac -version

Example output:

javac 21

If both commands work, the installation is successful.

---

5. Install an IDE

An **Integrated Development Environment (IDE)** helps developers write, debug, and run Java programs easily.

Popular Java IDEs include:

- IntelliJ IDEA
- Eclipse IDE
- Visual Studio Code

Why Use an IDE?

Benefits include:

- Code auto-completion

- Error detection
- Debugging tools
- Project management
- Integrated build tools

For beginners, **IntelliJ IDEA Community Edition** is often recommended.

---

6. Write and Run the First Java Program

Now let's create a simple Java program.

Step 1: Create a Java File

Create a file named:

HelloWorld.java

Step 2: Write the Code

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello, Java!");
   }
}
```

Step 3: Compile the Program

Open terminal and run:

```
javac HelloWorld.java
```

This creates a file:

HelloWorld.class

The .class file contains **Java bytecode**.

Step 4: Run the Program

Run the compiled program:

```
java HelloWorld
```

Output:

```
Hello, Java!
```

Java Program Execution Flow

HelloWorld.java
↓
javac (compiler)
↓
HelloWorld.class
↓
java (JVM)
↓
Program Output

Step	Description
Install JDK	Provides Java compiler and tools
Configure JAVA_HOME	Points to JDK installation
Configure PATH	Allows running Java commands
Verify installation	Check using java -version
Install IDE	Makes development easier
Write program	Create .java file and run it

Java Program Structure

A Java program follows a well-defined structure. Every Java application must contain at least **one class**, and program execution begins from the **main() method**. The Java program is compiled by the Java compiler and executed by the **Java Virtual Machine**.

Basic Structure of a Java Program

A simple Java program is written as follows

```
public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello Java");
  }
}
```

Components of a Java Program

A typical Java program consists of the following components:

1. Class Declaration
2. main() Method
3. Program Statements

Each component plays an important role in defining and executing the program.

Class Declaration

Every Java program must have at least **one class**.

Syntax:

Java

```
public class ClassName {

// class body

}
```

Show more lines

Key rules:

✓ File name must match the public class name
✓ One public class per file
✓ Classes contain methods, variables, constructors, etc.

Rules for Class Names

- Must start with a letter.
- Cannot contain spaces.
- Should follow **PascalCase naming convention**.

Examples:

StudentRecord
BankAccount
EmployeeDetails

The main() Method

The **main() method** is the entry point of a Java program. The JVM starts executing the program from this method.

Example:

public static void main(String[] args)

Explanation of Each Part

public :-

---

public is an access modifier in java. The main method must be declared as public otherwise JVM cannot execute our main method or in other words JVM can't enter inside the main method for execution of the program.

If main method is not declared as public then program will compile but it will not be executed by JVM.

Note :- From java compiler point of view there is no rule to declare our methods as public.

static :-

In java our main method is declared as static so we need not to create an object to call the main method.

If a static method is declared in the same class then from main method we can directly call that method, here object is not required.

If the static method is declared in another class then we can call that static method with the help of class name, here also object is not required.

If we don't declare the main method as static method then our program will compile but it will not be executed by JVM.

Points to remember :

---

1) Non-static method, we cannot call directly from main

method(Object is required)

2) static method declared in the same class, we can directly call

from main method

3) static method declared in another class, we can call with the

help of class name.

```
 class Labclass
{
   public static void main(String [] args)
   {
     Integer.parseInt(String x); //"123" --------> 123
   }
}
```

---

void :-

It is a keyword in Java. It means no return type. Whenever we define any method in java and if we don't want to return any kind of value from that particular method then we should write void before the name of the method.

Note :- In the main method if we don't write void or any other kind of return type then it will generate a compilation error.

In java whenever we define a method then compulsory we should define return type of method.(Syntax rule).

main() :-

It is a userdefined function/method because a user is responsible to define some logic inside the main method.main() method is very important method because every program execution will start from main() method only, as well as the execution of the program ends with main() method only.

Command Line Argument :-

Whenever we pass an argument/parameter to the main method then it is called Command Line Argument.The argument inside the main method is String because String is a alpha-numeric collection of character so, It can accept numbers,decimals, characters, combination of number and character.

That is the reason java software people has provided String as a parameter inside the main method.(More Wider scope to accept the values)

System.out.println() :-

It is an output statement in java, By using System.out.println() statement we can print anything on the console.

In System.out.println(), System is a predefined class available in java.lang package, out is a reference variable of PrintStream class available in java.io package and println() is a predfined method available in PrintStream class.

In System.out, .(dot) is a member access operator. It is called as period. It is used to access the member of the class.

In java we have print() and println() both methods are available, print() method will print the data and keep the cursor in the same line where as println() method will print the data and move the cursor to the next line.

3. Program Statements

Statements are the instructions that perform specific tasks in a Java program.

Example:

System.out.println("Hello Java");

This statement prints a message on the console.

Understanding System.out.println()

The output statement in Java is written as:

System.out.println("Hello Java");

This statement consists of three components:

System

System is a predefined class in the java.lang package that provides access to system-related resources.

out

out is a **static object of the PrintStream class**. It represents the standard output stream (console).

println()

println() is a method of the PrintStream class used to display text on the console and move the cursor to the next line.

Example:

System.out.println("Welcome to Java");

Output:

Welcome to Java

Compilation and Execution Process

Java programs follow a two-step process: **compilation** and **execution**.

Step 1: Writing the Source Code

The programmer writes the Java program and saves it with the .java extension.

Example:

HelloWorld.java

Step 2: Compilation

The Java compiler converts the source code into **bytecode** using the compiler provided in the **Java Development Kit**.

Command:

javac HelloWorld.java

This generates a file:

HelloWorld.class

The .class file contains bytecode instructions.

Step 3: Execution

The compiled bytecode is executed by the JVM using the following command:

java HelloWorld

Output:

Hello Java

Java Program Execution Flow

HelloWorld.java
↓
Java Compiler (javac)
↓
HelloWorld.class (Bytecode)
↓
Java Virtual Machine
↓
Program Output

Java Basics

Before writing complex Java programs, it is important to understand the basic building blocks of the language. These include **identifiers, keywords, reserved words, comments, naming conventions, and command-line arguments**.

These elements define how Java code is written, organized, and executed in the **Java Virtual Machine**.

---

1. Identifiers

An **identifier** is the name given to a variable, method, class, or any other program element.

Identifiers help programmers refer to program components.

Examples

```
int age;
String studentName;
class StudentRecord
```

Here:

- age → variable identifier
- studentName → variable identifier
- StudentRecord → class identifier

Rules for Identifiers

1. Must start with a **letter, underscore (_), or dollar sign ($)**.
2. Cannot start with a **number**.
3. Cannot use **Java keywords**.
4. Cannot contain **spaces**.
5. Identifiers are **case-sensitive**.

Valid Identifiers

```
age
studentName
_total
$amount
```

Invalid Identifiers

```
1name
student name
class
int
```

---

2. Keywords

Keywords are reserved words in Java that have predefined meanings.

They are used by the Java compiler to perform specific operations and **cannot be used as identifiers**.

Examples of Java keywords:

class, public, static, void, int, if, else, return, this, new, for, while

Example usage:

```
public class Test {
   int number;
}
```

In this example:

Here:

- public → access modifier keyword
- class → used to declare a class

---

3. Reserved Words

Reserved words include:

- **Java keywords**
- **Special literals**

Examples of reserved words:

true
false
null

Example:

```
boolean flag = true;
```

Here, true is a reserved literal.

---

4. Comments in Java

Comments are used to explain code and improve readability.
The compiler ignores comments during program execution.

Java supports three types of comments.

Single-Line Comment

Used for short explanations.

```
// This is a single-line comment
int age = 25;
```

Multi-Line Comment

Used for longer descriptions.

```
/*
This is a multi-line comment
It can span multiple lines
*/
```

Documentation Comment

Used to generate documentation using the Javadoc tool.

```
/**
 * This method calculates the sum of two numbers
 */
```

---

5. Naming Conventions

Java follows certain **naming conventions** to make code readable and maintainable.

Although not enforced by the compiler, these conventions are widely followed.

Class Names

Class names should start with a **capital letter** and follow **PascalCase**.

Examples:

```
Student
BankAccount
EmployeeDetails
```

Variable Names

Variable names should start with a **lowercase letter** and follow **camelCase**.

Examples:

studentName
totalMarks
accountBalance

Method Names

Method names also use **camelCase**.

Examples:

calculateTotal()
displayDetails()
getStudentName()

Constant Names

Constants are written in **UPPER_CASE with underscores**.

Examples:

MAX_VALUE
PI_VALUE
DEFAULT_SIZE

---

6. Command-Line Arguments

Command-line arguments allow users to pass input values to a Java program during execution.

These arguments are stored in the String[] args parameter of the main() method.

Example:

```
public class Test {

   public static void main(String[] args) {

      System.out.println(args[0]);

   }

}
```

Running the Program

java Test Java

Here:

args[0] = Java

Output:

Java

Example with Multiple Arguments

```
public class Sum {

   public static void main(String[] args) {
      int a = Integer.parseInt(args[0]);
      int b = Integer.parseInt(args[1]);
      System.out.println(a + b);

   }
}
```

Run command:

java Sum 10 20

Output:

30

1. Data Types in Java

Java provides two main categories of data types.

1. Primitive Data Types

Primitive data types store **simple values directly in memory**.

Characteristics:

- Predefined by Java
- Store actual values
- Faster memory access
- Fixed size

Java provides **8 primitive data types**.

Data Type	Size	Description	Example
byte	1 byte	small integer	byte b = 10;
short	2 bytes	small integer	short s = 200;
int	4 bytes	integer value	int num = 100;
long	8 bytes	large integer	long l = 100000L;
float	4 bytes	decimal number	float f = 10.5f;
double	8 bytes	large decimal	double d = 20.5;
char	2 bytes	single character	char c = 'A';
boolean	JVM dependent	true/false value	boolean flag = true;

✓ Stored in **stack memory**
✓ Hold simple, constant values
✓ Fast & efficient

int age = 25;

double price = 199.99;

char grade = 'A';

boolean isJavaFun = true;

. Reference (Non-Primitive) Data Types

Reference data types store **references (memory addresses)** of objects rather than the actual value.

Examples include:

- Classes
- Objects
- Arrays
- Strings
- Interfaces

Example:

```
String name = "Java";
int[] numbers = {1,2,3,4};
```

Here:

- name stores a reference to a **String object**
- numbers stores a reference to an **array objec**

Memory representation:

```
Stack Memory      Heap Memory

-------------     ----------------

name   --------->  "Java"

numbers --------->  [1,2,3,4]
```

Example:

Java

```
String name = "Revathi";
```

Show more lines

Here:

- "Revathi" → stored in **heap**
- name → reference stored in **stack**

What they store:

✓ Memory address of actual object
❌ Not the value directly

3. Variables

A **variable** is a named memory location used to store data.

Syntax:

```
datatype variableName = value;
```

Example:

```
int age = 30;
double salary = 50000.50;
```

Types of Variables

Local Variables

Declared inside a method or block.

Example:

```
public void display() {
   int number = 10;
}
```

Instance Variables

Declared inside a class but outside methods.

Example:

```
class Student {
   int id;
   String name;
}
```

Static Variables

Declared using the static keyword and shared by all objects.

Example:

```
class Counter {
   static int count = 0;
}
```

Variable Type	Scope	Lifetime
Local Variable	Inside method/block	During method execution
Instance Variable	Inside class	Until object is destroyed
Static Variable	Entire class	Until program ends

4. References

A **reference** is a variable that stores the **memory address of an object**.

Example:

String city = "Hyderabad";

Here:

- city is a reference variable
- It refers to a String object in heap memory.

Another example:

Student s = new Student();

Here:

- s stores the reference of a Student object.

---

5. Literals

A **literal** is a constant value assigned to a variable.

Example:

int x = 10;

Here:

- 10 is a literal.

Types of Literals

Integer Literals

int num = 100;

Floating-Point Literals

double price = 99.99;

Character Literals

char grade = 'A';

String Literals

String name = "Java";

Boolean Literals

boolean flag = true;

Type Casting

Type casting means **converting one data type into another data type**.

There are two types of type casting in Java.

6.1 Implicit Type Casting (Widening)

In widening, a **smaller data type is automatically converted into a larger data type**.

Example:

```
int num = 10;
double value = num;
```

Here:

int → double

Conversion hierarchy:

byte → short → int → long → float → double

6.2 Explicit Type Casting (Narrowing)

In narrowing, a **larger data type is converted into a smaller data type manually**.

Example:

```
double num = 10.5;
int value = (int) num;
```

Output:

10

Explanation:

- The decimal part is removed

Example:

```
public class TypeCastingDemo {
   public static void main(String[] args) {
      int a = 10;
      double b = a;  // widening
      double x = 20.5;
      int y = (int) x;  // narrowing
      System.out.println(b);
      System.out.println(y);
   }
}
```

Wrapper Classes in Java

In Java, **wrapper classes** are used to convert primitive data types into objects. Each primitive data type has a corresponding wrapper class defined in the **java.lang** package.

Wrapper classes allow primitive values to be used where objects are required, such as in **collections, generics, and object-based APIs**.

Example:

```
int num = 10;
Integer obj = Integer.valueOf(num);
```

Here:

- num → primitive value
- obj → wrapper object

Wrapper classes are part of the core Java language and are executed by the **Java Virtual Machine**.

Primitive Types and Wrapper Classes

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

These classes wrap primitive values inside objects.

Why Wrapper Classes Are Needed

Wrapper classes provide several advantages:

1. Allow primitives to be used as **objects**
2. Required for **Java Collections Framework**
3. Provide **utility methods**
4. Support **conversion between types**
5. Work with **generics**

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

Primitive int cannot be used directly, so the wrapper **Integer** is used.

Boxing and Unboxing

Wrapper classes support automatic conversion between primitive values and objects.

Autoboxing

Autoboxing converts a primitive type into its wrapper object automatically.

Example:

```
int a = 10;
Integer obj = a;
```

The compiler internally converts it to:

```
Integer obj = Integer.valueOf(a);
```

Unboxing

Unboxing converts a wrapper object into a primitive value.

Example:

```
Integer obj = 20;
int num = obj;
```

Internally:

```
int num = obj.intValue();
```

Common Methods in Wrapper Classes

Example using **Integer**

Method	**Description**
parseInt()	Converts string to int
valueOf()	Converts value to Integer object
toString()	Converts value to string
compareTo()	Compares values

Example:

```
String s = "100";

int num = Integer.parseInt(s);

System.out.println(num + 50);
```

Output

150

Type Casting in Java

Type casting is the process of converting one data type into another.

Java supports two types of casting:

1. Implicit casting (Widening)
2. Explicit casting (Narrowing)

Implicit Casting (Widening)

Implicit casting is done **automatically by the compiler** when converting a smaller data type into a larger data type.

Example:

```
int a = 10;
double b = a;
```

Conversion hierarchy:

byte → short → int → long → float → double

Example:

```
int number = 25;
double result = number;

System.out.println(result);
```

Output

25.0

Why it works:

- Larger data type can store the smaller value safely.

Explicit Casting (Narrowing)

Explicit casting is required when converting **a larger data type into a smaller data type**.

This conversion must be done manually.

Syntax:

```
datatype variable = (datatype)value;
```

Example:

```
double num = 10.5;
```

```
int result = (int) num;

System.out.println(result);
```

Output

10

Explanation:

- Decimal part is truncated.

Example of Data Loss

```
int num = 130;

byte b = (byte) num;

System.out.println(b);
```

Output

-126

Because byte range is:

-128 to 127

This causes **overflow**.

Type Promotion in Expressions

Java automatically promotes smaller types during arithmetic operations.

Example:

```
byte a = 10;
byte b = 20;

int result = a + b;
```

Even though a and b are bytes, the result becomes int.

Reason:

Arithmetic operations are executed in **int precision**.

Casting with Wrapper Classes

Example:

```
Integer obj = 100;

double value = obj.doubleValue();
```

Or:

```
String text = "50";

int num = Integer.parseInt(text);
```

Primitive vs Wrapper Comparison

Feature	Primitive	Wrapper
Type	Basic data type	Object
Memory	Stack	Heap
Performance	Faster	Slightly slower
Methods	Not available	Available

Example:

```
int a = 10;
Integer b = 10;
```

Performance Considerations

Frequent use of wrapper classes may cause:

- Additional memory usage
- Object creation overhead

Example:

```
Integer sum = 0;

for(int i=0;i<1000;i++){
   sum += i;
}
```

Better approach for heavy computation:

Use primitive types

Concept	Description
Wrapper Class	Object representation of primitive types
Autoboxing	Primitive → Object
Unboxing	Object → Primitive
Implicit Casting	Automatic conversion
Explicit Casting	Manual conversion
Type Promotion	Automatic upgrade during expressions

Operators in Java

Operators are special symbols used to perform operations on variables and values. Java provides various types of operators that allow programmers to perform arithmetic calculations, comparisons, logical operations, and assignments.

These operators are executed by the **Java Virtual Machine** during program execution.

Example:

```
int a = 10;
int b = 20;
int sum = a + b;
```

Here + is an **operator**, while a and b are **operands**.

Types of Operators in Java

Java operators are categorized into the following types:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Unary Operators
7. Ternary Operator

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b

```
public class ArithmeticDemo {
   public static void main(String[] args) {

      int a = 10;
      int b = 3;
      System.out.println(a + b);
      System.out.println(a - b);
      System.out.println(a * b);
      System.out.println(a / b);
      System.out.println(a % b);
   }
}
```

Output:

```
13
7
30
3
1
```

2. Relational Operators

Relational operators are used to compare two values.
The result of a relational operation is always **true or false**.

Operator	Description
==	Equal to
!=	Not equal to
>	Greater than
<	Less than

Operator	Description
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
int a = 10;
int b = 20;

System.out.println(a > b);
System.out.println(a < b);
System.out.println(a == b);
```

Output:

```
false
true
false
```

3. Logical Operators

Logical operators are used to perform logical operations on boolean expressions.

Operator	Description
&&	Logical AND
!	Logical NOT

Example:

```
boolean a = true;
boolean b = false;

System.out.println(a && b);
System.out.println(a || b);
System.out.println(!a);
```

Output:

```
false
true
false
```

4. Bitwise Operators

Bitwise operators perform operations on **individual bits of integer values**.

Operator	Description
&	Bitwise AND
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

Example:

```
int a = 5;
int b = 3;

System.out.println(a & b);
System.out.println(a | b);
System.out.println(a ^ b);
```

Binary representation:

5 = 101
3 = 011

Output:

1
7
6

5. Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Example	Meaning
=	a = 10	Assign value
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 2	a = a * 2
/=	a /= 2	a = a / 2

Operator	Example	Meaning
%=	a %= 2	a = a % 2

Example:

```
int a = 10;

a += 5;

System.out.println(a);
```

Unary Operators

Unary operators operate on **a single operand**.

Operator	Description
+	Unary plus
-	Unary minus
++	Increment
--	Decrement
!	Logical NOT

Example:

```
int a = 10;
a++;
System.out.println(a);
a--;
System.out.println(a);
```

Output:

```
11
10
```

Pre-Increment vs Post-Increment

Operator	Meaning
++a	Increment first, then use value
a++	Use value first, then increment

Example:

```
int a = 5;
System.out.println(++a);
System.out.println(a++);
System.out.println(a);
```

Output:

```
6
6
7
```

---

7. Ternary Operator

The **ternary operator** is a shorthand for the if-else statement.

Syntax:

```
condition ? expression1 : expression2
```

Example:

```
int a = 10;
int b = 20;

int max = (a > b) ? a : b;

System.out.println(max);
```

Output:

```
20
```

Explanation:

- If the condition is **true**, expression1 is executed.
- If the condition is **false**, expression2 is executed.

Operator Precedence

Operator precedence determines the **order in which operations are performed**.

Example:

```
int result = 10 + 5 * 2;
```

First multiplication happens, then addition.

Result:

```
20
```

Summary

Operator Type	Purpose
Arithmetic	Mathematical calculations
Relational	Comparison of values
Logical	Boolean operations
Bitwise	Bit-level operations
Assignment	Assign values to variables
Unary	Operations on single operand
Ternary	Short form of if-else

Control Flow Statements in Java

Control flow statements determine the **order in which program instructions are executed**. By default, Java executes statements sequentially from top to bottom. However, control flow statements allow developers to **change the normal execution flow** based on conditions or repeated execution.

Control flow mechanisms are executed by the **Java Virtual Machine** during runtime.

Control flow statements are broadly divided into three categories:

1. **Decision-making statements**
2. **Looping statements**
3. **Branching statements**

---

1. Decision-Making Statements

Decision-making statements allow the program to **execute different blocks of code depending on a condition**.

Java provides the following decision-making constructs:

- if
- if-else
- else-if

- switch

The if Statement

The if statement executes a block of code **only when a specified condition is true**.

Syntax

```
if(condition) {
  // code to execute
}
```

Example

```
public class IfExample {
  public static void main(String[] args) {

    int age = 20;

    if(age >= 18) {
      System.out.println("Eligible to vote");
    }

  }
}
```

Explanation

If the condition age >= 18 is true, the message is printed.

The if-else Statement

The if-else statement executes **one block if the condition is true and another block if the condition is false**.

Syntax

```
if(condition) {
  // true block
}
else {
  // false block
}
```

Example

```
int number = 10;
```

```
if(number % 2 == 0) {
   System.out.println("Even number");
}
else {
   System.out.println("Odd number");
}
```

The else-if Ladder

The else-if ladder is used when multiple conditions need to be evaluated.

Syntax

```
if(condition1) {
   // block1
}
else if(condition2) {
   // block2
}
else if(condition3) {
   // block3
}
else {
   // default block
}
```

Example

```
int marks = 75;

if(marks >= 90) {
   System.out.println("Grade A");
}
else if(marks >= 75) {
   System.out.println("Grade B");
}
else if(marks >= 50) {
   System.out.println("Grade C");
}
else {
   System.out.println("Fail");
}
```

2. The switch Statement

The switch statement is used to **select one of many possible execution paths** based on the value of an expression.

Syntax

```
switch(expression) {
  case value1:
    // statements
    break;

  case value2:
    // statements
    break;

  default:
    // default block
}
```

Example

```
int day = 3;

switch(day) {

  case 1:
    System.out.println("Monday");
    break;

  case 2:
    System.out.println("Tuesday");
    break;

  case 3:
    System.out.println("Wednesday");
    break;

  default:
    System.out.println("Invalid day");

}
```

Explanation

- break prevents execution from falling into the next case.
- default executes when no case matches.

3. Looping Statements

Loops allow a block of code to **execute repeatedly while a condition remains true**.

Java supports three looping statements:

- for loop
- while loop
- do-while loop

The for Loop

The for loop is used when the **number of iterations is known in advance**.

Syntax

```
for(initialization; condition; update) {
   // loop body
}
```

Example

```
for(int i = 1; i <= 5; i++) {
   System.out.println(i);
}
```

Output

```
1
2
3
4
5
```

Loop Components

Part	Description
Initialization	Variable initialization
Condition	Determines loop continuation
Update	Increment or decrement

The while Loop

The while loop executes a block of code **as long as the condition remains true**.

Syntax

```
while(condition) {
  // loop body
}
```

Example

```
int i = 1;

while(i <= 5) {
  System.out.println(i);
  i++;
}
```

The do-while Loop

The do-while loop executes the code **at least once**, even if the condition is false.

Syntax

```
do {
  // loop body
}
while(condition);
```

Example

```
int i = 1;

do {
  System.out.println(i);
  i++;
}
while(i <= 5);
```

Loop	Condition Check
for	Before execution
while	Before execution
do-while	After execution

Branching Statements

Branching statements alter the **normal flow of loop execution**.

Java provides:

- break
- continue
- return

The break Statement

The break statement **terminates the loop or switch statement immediately**.

Example

```
for(int i = 1; i <= 10; i++) {

  if(i == 5) {
    break;
  }

  System.out.println(i);
}
```

Output

```
1
2
3
4
```

The continue Statement

The continue statement **skips the current iteration and moves to the next iteration**.

Example

```
for(int i = 1; i <= 5; i++) {

  if(i == 3) {
    continue;
  }
```

```
  System.out.println(i);
}
```

Output

```
1
2
4
5
```

The return Statement

The return statement **terminates a method and optionally returns a value**.

Example

```
public int add(int a, int b) {

  return a + b;

}
```

Here, the method returns the sum of two numbers.

Statement	Purpose
if	Executes code when condition is true
if-else	Executes one of two blocks
else-if	Handles multiple conditions
switch	Multi-way decision statement
for	Loop with known iterations
while	Loop based on condition
do-while	Loop executed at least once
break	Terminates loop or switch
continue	Skips current iteration
return	Exits method and returns value

Arrays in Java

An **array** is a data structure that stores **multiple values of the same data type in a single variable**. Arrays help manage collections of data efficiently and allow access to elements using an **index**.

Arrays are objects managed by the **Java Virtual Machine** and are stored in **heap memory**.

Example:

int[] numbers = {10, 20, 30, 40, 50};

Here, numbers stores multiple integer values.

Array Fundamentals

Key characteristics of arrays:

- Stores **homogeneous elements** (same data type)
- Uses **index-based access**
- Array index starts from **0**
- Fixed size once created
- Stored in **heap memory**

Example:

int[] arr = new int[5];

This creates an array that can store **5 integers**.

Declaring and Creating Arrays

Declaration

datatype[] arrayName;

Example:

int[] numbers;

Array Creation

numbers = new int[5];

Declaration + Creation

int[] numbers = new int[5];